

Ethan Van Eyden (u25260244)

Task 1

1.1

Biomes – It is trying to be the abstract factory, and it should be abstract factory.

Map – It is trying to be the composite, and it should be the composite.

Place features – It is trying to be the decorator and it should be the decorator.

Place features – It is trying to be the decorator, and it should be the decorator.

Movement - It is trying to be the strategy, and it should be the state.

Routes - It is trying to be the state, and it should be the strategy.

The state pattern is swapped with the strategy pattern.

The intent of strategy is defining a set of algorithms and make them interchangeable.

Route selection is a perfect use case for this since it involves the same outcome (selecting a route) but the way it selects it depends on the priority.

The intent of state is to change the object based on its internal state. The vehicle choice and the consequences of picking a specific vehicle like costs depends on the objects internal state rather than what the client asks.

1.2

Strategy: Context, Strategy, ConcreteStrategy

Abstract Factory: AbstractFactory, ConcreteFactory, AbstractProduct, ConcreteProduct

State: Context, State, ConcreteState

Decorator: Component, ConcreteComponent, Decorator, ConcreteDecorator

Composite: Component, Leaf, Composite, Client

1.3

Fault 1: Abstract Factory Missing

- i. What's wrong: Biome creates products directly instead of using a factory hierarchy.
- ii. Rule broken: Separate product creation from product usage. Use parallel factory and product hierarchies.
- iii. Consequence: Adding a new biome requires modifying the Biome class. This violates the Open/Closed Principle.

Fault 2: Missing Virtual Destructors

- i. What's wrong: Base classes such as Map, TravelState, and RouteStrategy lack virtual destructors.
- ii. Rule broken: Any class intended for inheritance must have a virtual destructor.
- iii. Consequence: Deleting derived objects through base pointers can cause undefined behaviour and memory leaks.

Fault 3: Excessive Coupling

- i. What's wrong: One class knows about almost everything, including the traveller, map, biomes, routes, and features.
- ii. Rule broken: Classes should follow the Single Responsibility Principle and depend on abstractions rather than concrete implementations.
- iii. Consequence: Changes to one subsystem can require changes to the central class, making the system tightly coupled and difficult to maintain.

Fault 4: Multiplicity Omitted

- i. What's wrong: Associations in the UML diagram do not show multiplicities such as 1, *, or 0..1.
- ii. Rule broken: UML associations should clearly communicate how many objects can participate in a relationship.

- iii. Consequence: The relationships are ambiguous. For example, it is unclear whether a Region can contain zero or many Locations, or whether a Location can belong to multiple Regions.

Fault 5: No Virtual Destructors

- i. What's wrong: Some base classes do not declare virtual destructors even though they are used polymorphically.
- ii. Rule broken: A polymorphic base class should have a virtual destructor so derived objects can be destroyed correctly through base-class pointers.
- iii. Consequence: Deleting an object through a base pointer can result in undefined behaviour and resources not being released correctly.

Fault 6: Region Uses Aggregation Instead of Composition

- i. What's wrong: Region uses a hollow diamond (aggregation) to Map instead of a filled diamond (composition).
- ii. Rule broken: In the Composite pattern, the composite should own its children when it is responsible for deleting them.
- iii. Consequence: Ownership is unclear, which can lead to memory leaks, dangling pointers, or multiple objects incorrectly managing the same child.

Fault 7: Decorator Uses Aggregation Instead of Composition

- i. What's wrong: The decorator uses a hollow diamond (aggregation) to Map instead of a filled diamond (composition).
- ii. Rule broken: The decorator should own the object it wraps when it is responsible for deleting it.
- iii. Consequence: The wrapped object may never be deleted when the decorator is destroyed, causing a memory leak.

Fault 8: Concrete Class Dependencies

- i. What's wrong: Traveller depends directly on concrete states such as Walk, Rover, and Ship instead of depending on the TravelState abstraction.
- ii. Rule broken: Depend on abstractions rather than concrete implementations.
- iii. Consequence: Adding a new travel mode can require changes to Traveller, increasing coupling and making the system harder to extend.

1.4

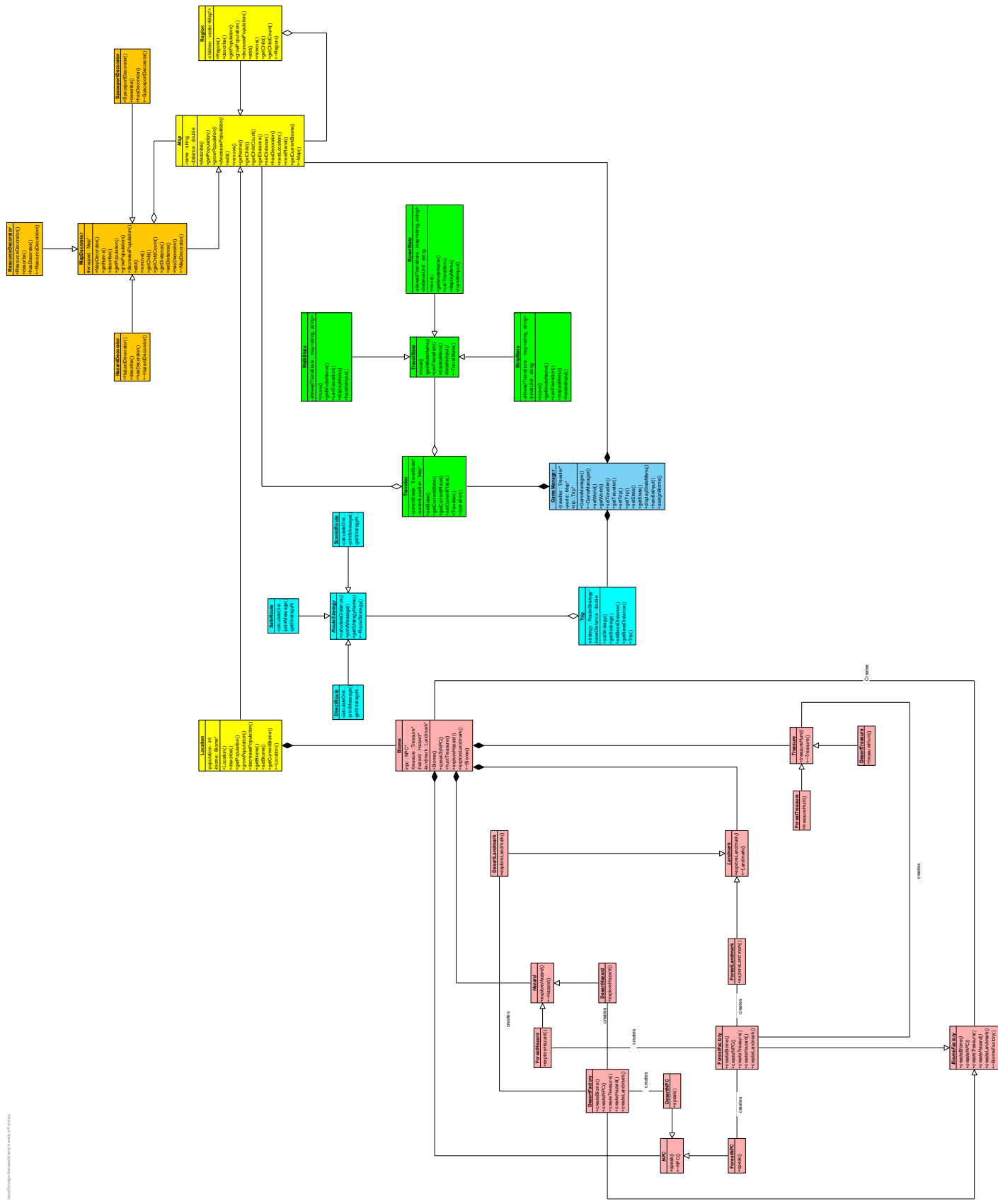
Worldbuilder is given access to the concrete products, and it should not know about their existence. The purpose of abstract factory is to decouple the creation of products from the client without knowing the exact product it will receive. The abstract factory is not supposed to know about the concrete products as something could break when the client gave unrelated strings in the make method you can products returned that are not supposed to be returned together.

1.5

The diagram is difficult to read because all five subsystems are mixed together, relationships cross each other, and the pattern participants are not visually separated. It is also unclear which classes are abstract, which are concrete, and which relationships represent ownership or dependency. I would improve the layout by grouping the classes into five labelled subsystems, placing abstract classes/interfaces above their concrete implementations, reducing crossing lines, and using consistent UML notation for inheritance, composition, aggregation and dependencies. Only important collaborations between subsystems should cross the subsystem boundaries.

Task 2

2.1



2.2

Strategy: The participants are Context, Strategy, and ConcreteStrategy. The Trip acts as the Context, while the different movement behaviours are represented by concrete strategies. This separates the algorithm for performing an action from the object using it, allowing movement behaviour to be changed without modifying the Traveller.

Abstract Factory: The participants are AbstractFactory, ConcreteFactory, AbstractProduct, and ConcreteProduct. The concrete biome factories are responsible for creating the corresponding biome objects, such as NPCs, hazards, treasures, and landmarks. The factory owns the creation of these related objects so that the client does not need to depend directly on their concrete classes.

State: The participants are Context, State, and ConcreteState. Traveller acts as the Context, while ShipState, RoverState, and WalkState are concrete states. The current state is owned by the Traveller because the traveller's behaviour and available actions depend on its current mode of movement. This allows state transitions to occur without placing all state-specific behaviour into one large class.

Decorator: The participants are Component, ConcreteComponent, Decorator, and ConcreteDecorator. The decorator structure is used to dynamically add properties or requirements to world locations. The decorator owns a reference to the wrapped component rather than inheriting its implementation, allowing multiple decorators to be combined without creating a separate subclass for every possible combination.

Composite: The participants are Component, Leaf, Composite, and Client. The world map is represented as a hierarchy of composite objects containing smaller areas and individual locations. The composite owns its child components, allowing the client to treat individual locations and groups of locations uniformly. This makes the hierarchical structure easier to traverse and extend.

2.3

1. Forest Biome — Abstract Factory

We added a Forest biome with its own NPC, hazard, treasure and landmark, including a Forest NPC, carnivorous flora, a research cache and an ancient bioluminescent tree.

This extends the Abstract Factory pattern by introducing a new family of concrete products. The forest objects implement the existing product abstractions, allowing the new biome to be added without modifying the existing concrete factories or client code.

2. Desert Biome — Abstract Factory

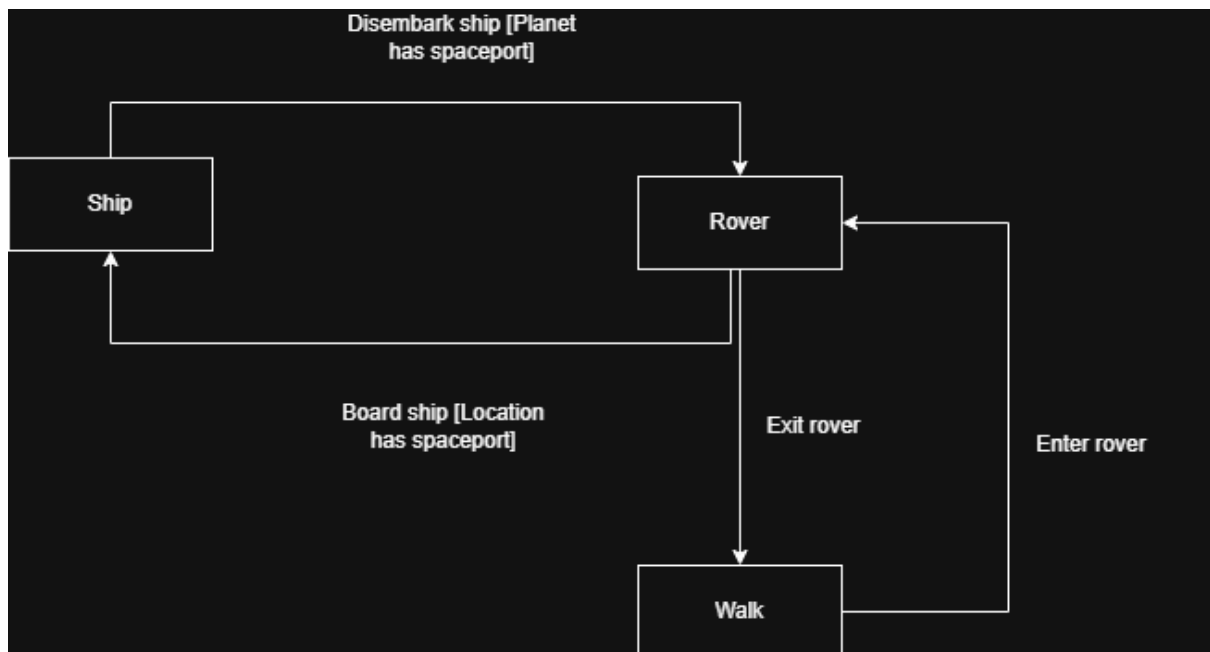
We added a Desert biome containing a Desert NPC, sandstorm hazard, buried supply pod and fallen starship landmark. This extends the Abstract Factory pattern with another concrete product family. The new factory produces the required desert objects through the existing abstractions, so the existing biome implementations do not need to be changed.

3. Ship, Rover and Walk Travel Modes — State

We introduced three different travel modes using ShipState, RoverState and WalkState. These extend the State pattern by encapsulating the movement rules and available commands for each mode in separate concrete state classes. The Traveller interacts with the states through the common State abstraction, allowing the travel modes to be changed or extended without modifying the existing movement implementations. In the ship state you can move between planets, in the rover state you can move between locations on planets and the walk state allows you to explore each location's biome.

Task 3

3.3



Task 4

4.3

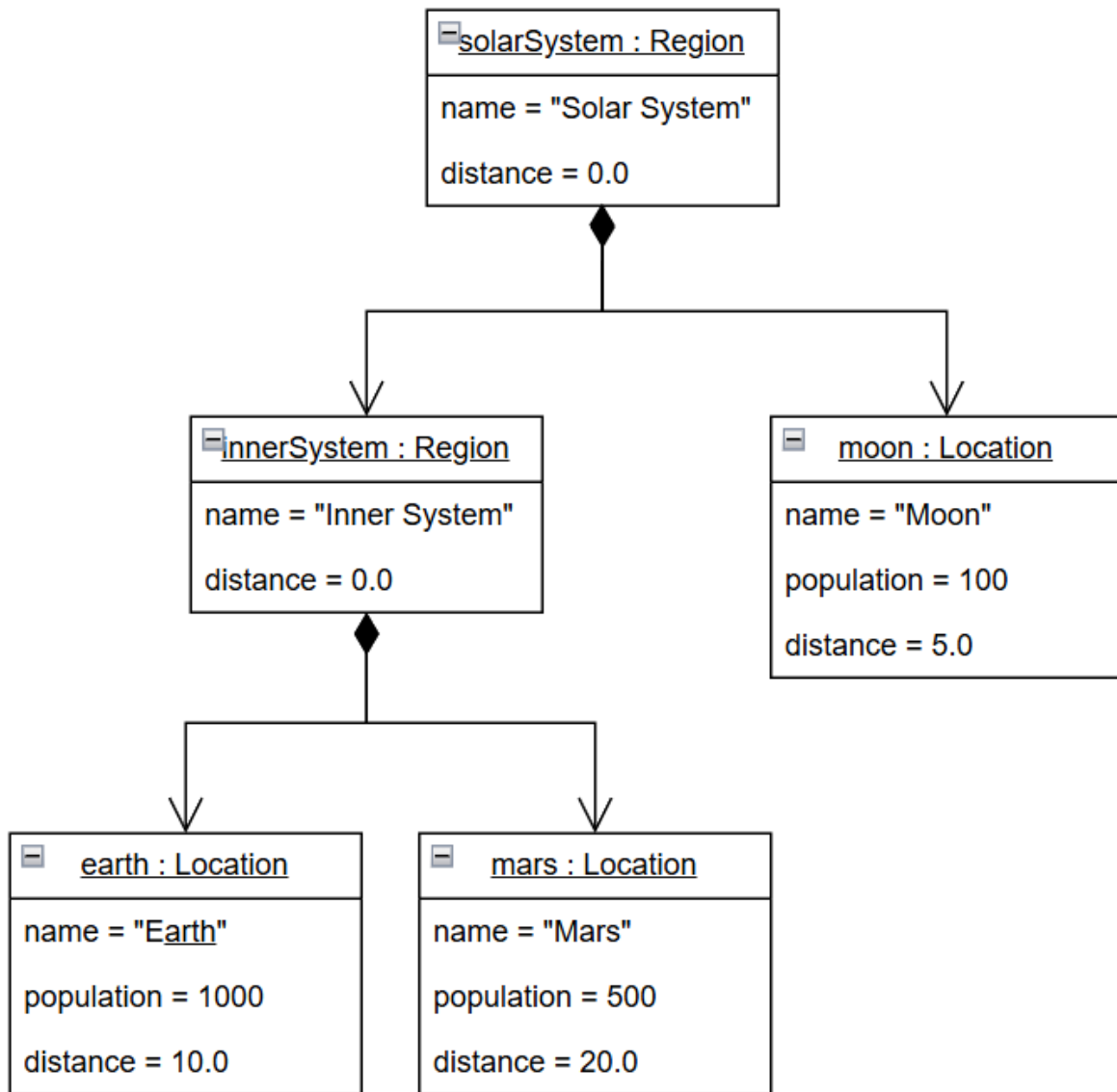
Routes use Strategy because they provide different algorithms for solving the same problem: finding a route between locations. A route strategy can be selected or replaced without changing the state or identity of the traveller.

Movement uses State because the traveller's behaviour depends on its current mode. ShipState, RoverState and WalkState determine which movement operations are available, and the traveller can transition between these states.

Although both patterns use an abstraction with concrete implementations, their intents differ. **Strategy represents a replaceable choice of algorithm, while State represents a changing condition that alters an object's behaviour.** Therefore, the two designs are not interchangeable: route algorithms are choices made by the client, whereas movement states represent the traveller's current mode.

Task 5

5.2



Task 6

